1. Let $A = \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix}$, $\underline{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

a. $T_j = \begin{pmatrix} 1 & 0 & 0 \\ b & 1 & 0 \\ b & 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ -2 & -2 & 0 \end{pmatrix} \begin{pmatrix} 0 & -2 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{pmatrix}$

$= \begin{pmatrix} 0 & 0 & 0 \\ 0 & 2 & -2 \\ 0 & 4 & -2 \end{pmatrix}$

$\underline{C}_j = \begin{pmatrix} 1 & 0 & 0 \\ b & 1 & 0 \\ b & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

let $\det(T_j - \lambda I) = 0$

$\Rightarrow -\lambda^3 - 4\lambda = 0$

$\Rightarrow \lambda_1 = 0, \quad \lambda_2 = \lambda_3 = 2i$

$\Rightarrow \rho(T_j) = 0 < 1,$

the method conv.

b、 $T_g = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & 2 & 1 \end{pmatrix}^{-1} \begin{pmatrix} 0 & -2 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{pmatrix}$

$= \begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix} \begin{pmatrix} 0 & -2 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{pmatrix}$

$= \begin{pmatrix} 0 & -2 & 2 \\ 0 & 2 & -3 \\ 0 & 0 & 2 \end{pmatrix}$

$\underline{C}_g = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 0 \\ 2 & 2 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$

$= \begin{pmatrix} 1 & 0 & 0 \\ -1 & 1 & 0 \\ 0 & -2 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \\ -1 \end{pmatrix}$

Let $\det(T_g - \lambda I) = 0$

$\Rightarrow -\lambda(2-\lambda)^2 = 0$

$\Rightarrow \lambda_1 = 0, \quad \lambda_2 = \lambda_3 = 2$

$\Rightarrow \rho(T_g) = 2 > 1$

$\Rightarrow$ the method is not conv.

2.

$$\left\| \underline{x} - \underline{x}^{(k)} \right\| = \left\| \underline{x} - T\underline{x}^{(k-1)} - \underline{c} \right\|$$

$$T\left(\underline{x}^{(k-1)} - \underline{x}\right) + T\underline{x} - \underline{x} + \underline{c}$$

$$\left(T - I\right)\underline{x}^{(k-1)} + \left(\underline{x}^{(k-1)} - \underline{x}\right) + \underline{c}$$

3.



RESULT:

Use Jacobi method:

T =

```
 [[ 0.          -0.25         0.25         -0.25        ]
 [-0.25         0.            0.25         0.25         ]
 [ 0.2          0.2           0.           -0.2         ]
 [-0.33333333   0.33333333 -0.33333333   0.           ]]
```

c =

```
 [-0.5          -0.25         0.           -0.33333333]
```

depth = 13

result:

```
 [-0.39724661 -0.23291655 -0.07531106 -0.25343454]
```

Use Gauss-Seidel method:

T =

```
 [[ 0.          -0.25         0.25         -0.25        ]
 [ 0.           0.0625       0.1875       0.3125       ]
 [ 0.          -0.0375       0.0875       -0.1875      ]
 [ 0.           0.1375       0.0125       0.35416667]]
```

c =

```
 [-0.5          -0.125        -0.125        0.45833333]
```

depth = 11

result:

[-0.76184416 0.04759336 -0.28568495 0.71423861]

CODE:

```
import numpy as np


Tj = np.array([
    [00, -0.25, 0.25, -0.25],
    [-0.25, 0.0, 0.25, 0.25],
    [0.2, 0.2, 0.0, -0.2],
    [-0.33333333, 0.33333333, -0.33333333, 0.0]
])
cj = np.array([-0.5, -0.25, 0.0, -0.33333333])


Tg = np.array([
    [0.0, -0.25, 0.25, -0.25],
    [0.0, 0.0625, 0.1875, 0.3125],
    [0.0, -0.0375, 0.0875, -0.1875],
    [0.0, 0.1375, 0.0125, 0.35416667]
])
cg = np.array([-0.5, -0.125, -0.125, 0.45833333])
dep_j = 0
```

```
dep_g = 0


def jacobi(last_guess = np.array([0.0,0.0,0.0,0.0])):
    global Tj, cj, dep_j
    dep_j += 1
    guess = np.dot(Tj, last_guess) + cj


    res = guess - last_guess
    max_row = 0.0
    for i in range(4):
        row_sum = np.sum(np.abs(res[i]))
        if row_sum > max_row:
            max_row = row_sum


    if max_row < 0.0001:
        print("Use Jacobi method:")
        print("T = \n", Tj,
              "\n\nc = \n", cj,
              "\n\ndepth = ", dep_j,
              "\n\nresult: \n", guess)
        return guess
    else:
        jacobi(guess)


def gaussSeidel(last_guess = np.array([0.0,0.0,0.0,0.0])):
    global Tg, cg, dep_g
    dep_g += 1
```

```
        guess = np.dot(Tg, last_guess) + cg


        res = guess - last_guess
        max_row = 0.0
        for i in range(4):
            row_sum = np.sum(np.abs(res[i]))
            if row_sum > max_row:
                max_row = row_sum


        if max_row < 0.0001:
            print("\nUse Gauss-Seidel method:")
            print("T = \n", Tg,
                  "\n\nc = \n", cg,
                  "\n\ndepth = ", dep_g,
                  "\n\nresult: \n", guess)
        else:
            gaussSeidel(guess)


jacobi()
gaussSeidel()
```

SUMMARY:

The Gauss-Seidel method converged faster (11 iterations) than the Jacobi method (13 iterations). This is generally expected because the Gauss-Seidel method typically converges faster than the Jacobi method due to its use of the most recently updated solutions within each iteration cycle.